FEVtools

User Guide

Version 1.1.0

Version date: Xxxxx x, 2026
Manual date: March 6, 2026

Contents

1 The Emedding Tensor and the Flag-Graph **3**

2 File formats **5**
2.1 The .fev format 5
2.2 The .flg format 5
2.3 The .fcs format 6

3 Using FEVtools **10**
3.1 The tools 10
3.1.1 Compilation 11
3.1.2 A few words about input files 11
3.2 Conversion tools 12
3.3 Creating an illustrative figure of the tensor 13
3.4 Computing sum rules for an embedding tensor 13
3.5 Identifying the symmetry group 13
3.6 Polygonal symbol generation 14
3.7 Reduction to primitive cell 15
3.8 Isomorphism checking 15
3.9 Structure generation 15
3.10 Coordinates generation 16
3.11 Files generation from coordinates 20

1 The Emedding Tensor and the Flag-Graph

The *Embedding Tensor* (*fev tensor*) consists of a cuboid data structure Φ with elements $\Phi_{f,e,v}$, where the $f/e/v$ indexes label the faces/edges/vertices of the system. These elements are given by:

$$\Phi_{f,e,v} = \begin{cases} 1, & \text{if vertex } v \text{ is contained in edge } e \\ & \text{and edge } e \text{ is contained in face } f\text{, and} \\ 0, & \text{otherwise.} \end{cases} \tag{1.1}$$

The labelling of the face/edge/vertex objects is illustrated for the structure in Fig. 1.1a, where we label the 6 vertices by black color numbers, the 9 edges by blue color integers and the 3 faces by red color values. Note that we must obey periodic boundary conditions, so that the four octagons centered on the corners of the unit cell are actually the same face (face 3), as well as the two squares cut by the horizontal limits of the rectangular cell (face 2). This also applies to the edges, as edge 9 cuts both vertical limits of the unit cell, similarly to edges 7 and 8, that cut both horizontal boundaries. The embedding tensor for this structure is illustrated in Fig. 1.1b. Note, for instance, that $\Phi_{3,7,1} = 1$ (see green circle in Fig. 1.1b), as vertex 1 and edge 7 both make part of face 3, as well as edge 7 contains vertex 1.

Each nonzero element of the embedding tensor defines an entity called a *flag*. When the involved edge is a bridge, each nonzero tensor element defines a pair of flags, usually called *twin flags*. A flag can be represented graphically over the structure by an arrow that emerges from the corresponding vertex, along its corresponding edge, and has a mark (two small parallel segments) towards the corresponding face. It also defines either a clockwise or a counterclockwise orientation around the face. In Fig. 1.1 we illustrate the $(1, 2, 3)$ flag, where the blue color of the flag symbol and the blue clockwisely oriented circle indicate its orientation. In Fig. 1.1d we illustrate the counterclockwisely oriented $(1, 4, 4)$ flag in red color.

Each flag has neighboring flags, with which they share two elements in common. Each flag has a face-neighbor (same e and v), an edge-neighbor (same f and v), and a vertex-neighbor (same f and e). When we connect each flag to its neighbors, we have what we define as the flag-graph. Twin flags have the same three defining elements (same f, e, and v), but are considered face-neighbors for each other by convention.

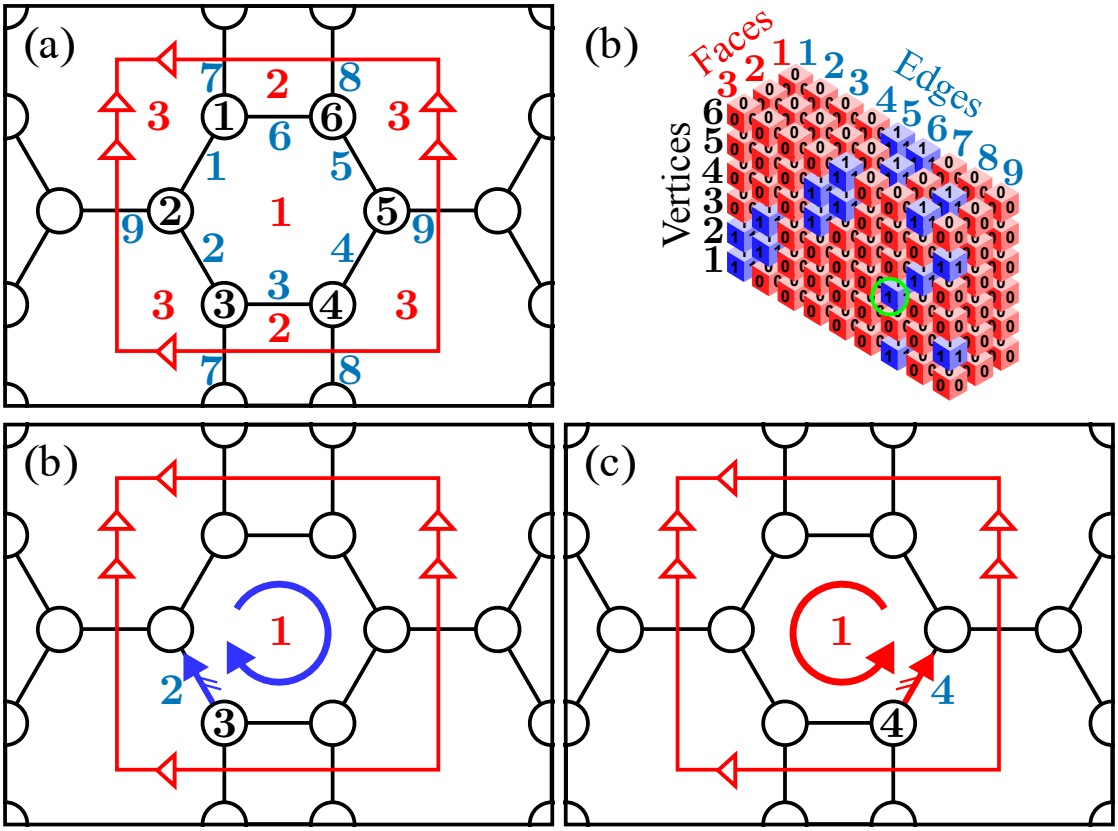

Figure 1.1: (a) Illustration of how faces (red numbers), edges (blue numbers), and vertices (black numbers) are labelled over a biphenylene periodic system. (b) Tensor cuboid data structure for biphenylene, where $\Phi_{f,e,v} = 1/\Phi_{f,e,v} = 0$ entries are represented by blue/red cubes. The green circle highilights the $\Phi_{3,7,1} = 1$ element. (c) Graphical representation of the $(1, 2, 3)$ flag, and its clockwise orientation. (d) Graphical representation of the $(1, 4, 4)$ flag, and its counterclockwise orientation.

2 File formats

The embedding tensor is the basic data structure in FEVtools. However, there are more self-contained structures that are more convenient for certain tasks involving the studied structures. In this chapter we describe in details each of these formats.

2.1 The .fev format

In FEVtools, the embedding tensor is exchanged with the user through a `.fev` file. This format is decribed below:

- A first line containing three integers, `nf`, `ne`, and `nv`, corresponding to the number of faces, edges and vertices of the tensor, respectively;
- One line for each tensor entry (either 0 or 1), where the elements are ordered in a nested triple-loop, where the innermost (outermost) loop runs over the vertices (faces).

A simple FORTRAN code reads the tensor from a `.fev` file with `UNIT=1` as illustrated below:

```
READ(1,*) nf,ne,nv          ! .fev file format:
DO i=1,nf                   ! - 1st line: nf ne nv
  DO j=1,ne                 ! - Then one line per each tensor entry (0 or 1)
    DO l=1,nv               ! -> Outer loop --> Faces
      READ(1,*) fev(i,j,l)  ! -> Middle loop -> Edges
    END DO                  ! -> Inner loop --> Vertices
  END DO                    !
END DO
```

The `.fev` format is an easy way for the user to input the studied structure. Other formats are more adequated to obtain further information, but they can be extracted from the `.fev` file from specific tools.

2.2 The .flg format

A flag is defined as a (f, e, v) integer trio for which $\Phi_{f,e,v} = 1$ (eventually we can have twin flags with $(f', e', v') = (f, e, v)$). They form a *flag-graph*, an entity that can be extracted from a given embedding tensor, and consists of: 1) the list of flags, 2) the list of face-, edge-, and vertex-neighbors of each flag, and 3) the flags orientations. In FEVtools,

the flag-graph of a system can be exchanged with the user through a `.flg` file. This format is decribed below:

- A first line containing four integers, `nflags`, `nf`, `ne`, and `nv`, corresponding to the number of flags, faces, edges and vertices of the stucture, respectively;
- A second line containing a vector with the number of edges from each face of the structure.
- In the following we have one line per flag, containing 7 integers: 1-3) the f, e, and v elements of the flag, 4-6) its face-, edge-, and vertex neighbor flags, and 7) the flag orientation (either +1 or -1).

In FORTRAN it can be read as:

```
READ(1,*) nflags,nf,ne,nv
READ(1,*) nface
DO i=1,nflags
  READ(1,*) flag(i,:),neigh_flag(i,:),flag_color(i)
END DO
```

where `flag` and `neigh_flag` are `nflag`×3 matrices. An alternative `.b.flg` format is written/read in binary form in FORTRAN as

```
READ(1) nflags,nf,ne,nv
READ(1) nface
READ(1) flag
READ(1) neigh_flag
READ(1) flag_color
```

The `.flg` format is convenient to find symmetry maps that enable us to determine the symmetry group of a given structure or to determine if tow structures are equivalent or not.

2.3 The .fcs format

The *faces-info* data structure is the most complete one when using FEVtools. It contains structural information relative to each face of the system. It further classifies faces and edges into *representative* of *non-representative*. In a set of faces which are symmetry-related, we elect only one of them to be the *representative* one, while the reamining ones are classified as *non-representative*. The same classification is applied to the edges within a given face: if there are symmetry-related edges within a given face, one is elected as the *representative*, with the other ones being called *non-representatives*. The *faces-info* data structure further indicates explicitly which edges are bridges. In FEVtools, the *faces-info* data can be exchanged with the user through a `.fcs` file format, as decribed below:

- A first line containing four integers, `nf`, `ne`, `nv`, and `nmax`, corresponding to the number of faces, edges, vertices of the structure, and the maximal number of edges in a face, respectively;
- Then we have blocks of 6 lines, one for each face of the structure. Each block contains:
  1. The number of edges in the face and a logical (`T` or `F`) value indicating if the face is a representative face (`T`) or if it is non-representative one (`F`);
  2. The faces touching the reference face;
  3. The edge forming the reference face;
  4. The vertices forming the reference face;
  5. Logical entries indicating if the edges forming the face are bridges or not;
  6. A set of logical values (one for each edge of the face) indicating if the edge is a representative one within the face (`T`) or if it is a non-representative one (`F`).

The vertices forming the reference face `i` are stored in a `v_in_f(i,j)` vector, in which `j` runs from 1 to the face size `n`. The same applies to the edges forming face `i`, which are stored in the `e_in_f(i,j)` vector, and to the faces touching face `i`, stored in the `f_in_f(i,j)` vector, always with `j=1,...,n`. All the elements in `f_in_f`, `e_in_f`, and `v_in_f` vectors have to be ordered around the face. This is done in such a way that the `j`-th edge from face `i` starts in vertex `v_in_f(i,j)` and finishes in vertex `v_in_f(i,j+1)`, while the `j=n`-th edge (`n` the size of face `i`) starts in vertex `v_in_f(i,n)` and finishes in vertex `v_in_f(i,1)`. The `j`-th edge (`e_in_f(i,j)`) is also the frontier between faces `i` and `f_in_f(i,j)`.

For instance, look at the example from Fig. 2.1a, where the reference face is `i=7`. Its starting elements are `f_in_f(7,1)=4`, `e_in_f(7,1)=7`, and `v_in_f(7,1)=2` (elements within the green curve). In the following, we have `f_in_f(7,2)=6`, `e_in_f(7,2)=1`, and `v_in_f(7,2)=5` (elements within the orange path). Explicitly, we can write

```
v_in_f(7,1:8) = [2, 5, 3, 7, 1, 6, 8, 4]
e_in_f(7,1:8) = [7, 1, 8, 2, 4, 6, 5, 3]
f_in_f(7,1:8) = [4, 6, 9, 8, 3, 1, 5, 2]
```

In this example, no edges are bridges, so that

```
b_in_f(7,1:8) = [F, F, F, F, F, F, F, F]
```

Suppose only edges 4 and 7 are equivalent by symmetry, then `u_in_f(7,1)=T` (relative to edge `e=7`) and `u_in_f(7,5)=F` (relative to edge `e=4`), since edge 7 is the first, and edge 4 is the fifth to appear in the sequence around the face (`u_in_f(7,j)=T` for all the other `j` values different from 1 and 5, since there are no more equivalent edges). Explicitly: In this example, no edges are bridges, so that

```
u_in_f(7,1:8) = [T, T, T, T, F, T, T, T]
```

Let us now consider the example from Fig. 2.1b, which features a bridge. In this case, edge `e=2` appears twice around `f=7`, and we have

```
v_in_f(7,1:8) = [2, 5, 3, 2, 4, 6, 1, 4]
e_in_f(7,1:8) = [7, 1, 3, 2, 4, 6, 5, 2]
f_in_f(7,1:8) = [4, 6, 2, 7, 3, 1, 5, 7]
```

Since edge 2 is a bridge, `b_in_f(7,4)=T` and `b_in_f(7,8)=T`, while `b_in_f(7,i)=F` for the other values. As they are the same edge, `e_in_f(7,4)=e_in_f(7,8)=2` are equivalent so that `u_in_f(7,4)=T` and `u_in_f(7,8)=F` since it appears firstly on the 4-th position. Explicitly

```
b_in_f(7,1:8) = [F, F, F, T, F, F, F, T]
u_in_f(7,1:8) = [T, T, T, T, T, T, T, F]
```

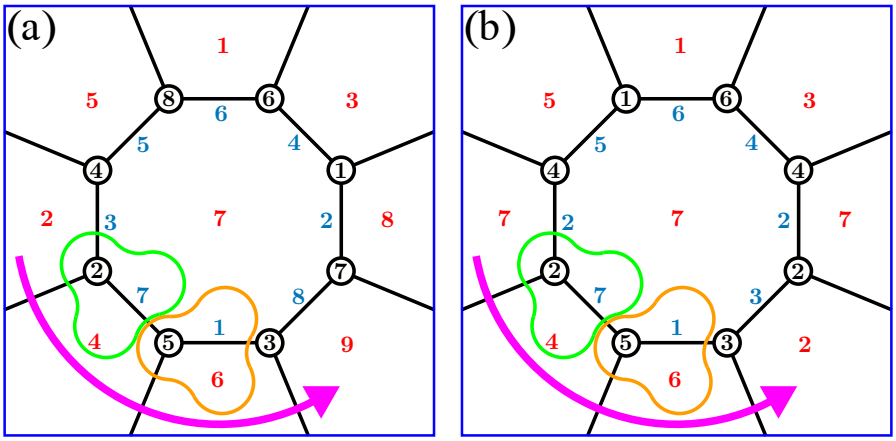

Figure 2.1: (a) Illustration of the sequence of neighboring faces, and composing edges and vertices of a given face (face 7). For the face `i=7` with size `n=8`, `v_in_f(7,1:8)` contains 2, 5, 3, 7, 1, 6, 8, and 4 in this order, while `e_in_f(7,1:8)` contains 7, 1, 8, 2, 4, 6, 5, and 3, as well as `f_in_f(7,1:8)` contains 4, 6, 9, 8, 3, 1, 5, and 2. The green curve highilights the `f_in_f(7,1)=4`, `e_in_f(7,1)=7`, and `v_in_f(7,1)=2` elements, while the orange curve highlights the `f_in_f(7,2)=6`, `e_in_f(7,2)=1`, and `v_in_f(7,2)=5` elements. (b) Same as (a), but an example with a bridge, where `v_in_f(7,1:8)` contains 2, 5, 3, 2, 4, 6, 1, and 4, `e_in_f(7,1:8)` contains 7, 1, 3, 2, 4, 6, 5, and 2, and `f_in_f(7,1:8)` contains 4, 6, 2, 7, 3, 1, 5, and 7.

In FORTRAN it can be read as:

```
READ(1,*) nf,ne,nv,nmax
DO i=1,nf
  READ(1,*) nface(i),uneq_face(i)
  READ(1,*) f_in_f(i,1:nface(i))
```

```
  READ(1,*) e_in_f(i,1:nface(i))
  READ(1,*) v_in_f(i,1:nface(i))
  READ(1,*) b_in_f(i,1:nface(i))
  READ(1,*) u_in_f(i,1:nface(i))
END DO
```

where `f_in_f` is a `nf`×`nmax` matrix, just as `e_in_f` and the others. An alternative `.b.fcs` format is written/read in binary form in FORTRAN as

```
READ(1) nf,ne,nv,nmax
DO i=1,nf
  READ(1) nface(i),uneq_face(i)
  READ(1) f_in_f(i,1:nface(i))
  READ(1) e_in_f(i,1:nface(i))
  READ(1) v_in_f(i,1:nface(i))
  READ(1) b_in_f(i,1:nface(i))
  READ(1) u_in_f(i,1:nface(i))
END DO
```

The `.fcs` format is convenient to conduct structure generation (and coordinates generation) through an add dimer approach, as well as the natural format to determine the polygonal symbol of the system.

3 Using FEVtools

FEVTOOLS is a FORTRAN package of applications that includes a total of 16 tools. Some of these tools are devoted to format conversion involving the `.fev`, `.flg`, and `.fcs` types. The remaining ones are intended to 9 different tasks, as listed below:

- `pov`: tensor illustration generation in POVRAY;
- `sum`: computation of sum rules;
- `gen`: structure generation;
- `smb`: polygonal symbol generation;
- `xyz`: coordinates generation (or `.fxx` files generation from coordinates);
- `iso`: isomorphism checking;
- `pri`: primitive cell reduction;
- `sym`: symmetry identification.

A fluxogram detailing the tools and their purposes is illustrated in Fig. 3.1.

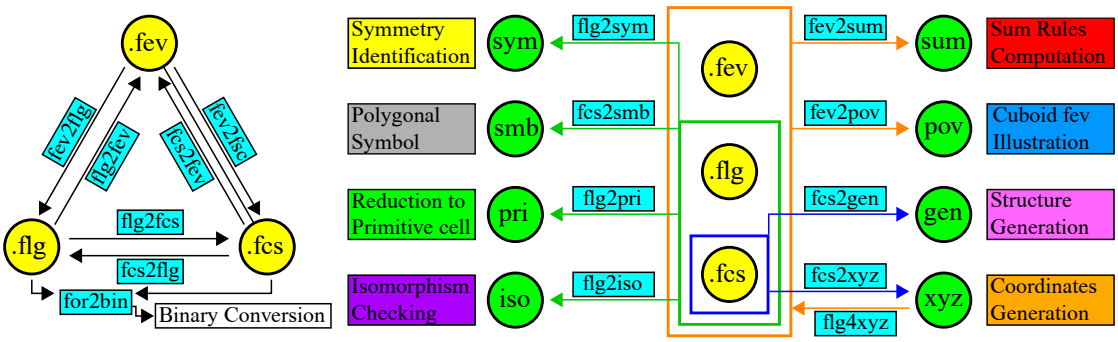

Figure 3.1: Summary of the utilities of the FEVTOOLS package.

3.1 The tools

The 16 FEV-tools are explicitely listed below:

- `fev2fcs`: tensor to faces-info conversion;

- `fev2flg`: tensor to flag-graph conversion;
- `fcs2fev`: faces-info to tensor conversion;
- `fcs2flg`: faces-info to flag-graph conversion;
- `flg2fcs`: flag-graph to faces-info conversion;
- `flg2fev`: flag-graph to tensor conversion;
- `for2bin`: formatted to unformatted format conversion (and vice-versa);
- `fev2pov`: Tensor illustration generation in POVRAY from embedding-tensor;
- `fev2sum`: computes sum rules from the embedding-tensor;
- `fcs2gen`: structure generation;
- `fcs2smb`: polygonal symbol generation from faces-info;
- `fcs2xyz`: coordinates generation;
- `flg2iso`: isomorphism checking from flag-graph;
- `flg2pri`: reduction to primitive cell from flag-graph;
- `flg2sym`: computes the symmetry group from flag-graph;
- `flg4xyz`: computes fev-tensor/flag-graph/faces-info from coordinates.

3.1.1 Compilation

To compile the package in a simple LINUX environment, just edit the `makefile` indicating your FORTRAN compiler, enter in the package folder through the terminal, and type `make` to build all the 16 tools. Each tool is used by simply executing the corresponding binary in a terminal by typing "`./binary`".

3.1.2 A few words about input files

Except for `fcs2gen` and `flg2iso`, all the tools deal with a single system and read an `inp` file which contains a `systemlabel` string in its first line. Such a string identifies the system being studied, with the structure's relevant input file being named as a `systemlabel.fev`, `systemlabel.flg`, `systemlabel.b.flg`, `systemlabel.fcs` or a `systemlabel.b.fcs` file, depending on the tool in consideration. The `flg2iso` tool deals with a pair of systems, whose `systemlabel1` and `systemlabel2` labels are also read from an `inp` file (one system string per line). On the other hand, the `fcs2gen` tool does not read an `inp` file, but reads an integer (from an `igen` file) which represents the previous generation of structures. We detail the use of each tool in the next sections.

3.2 Conversion tools

There are 7 tools for format conversion involving the `.fev`, `.flg`, and `.fcs` formats. Upon simple execution, all of them read a `inp` file containing only the `systemlabel` string in the first line, and work on `systemlabel.b.***` or `systemlabel.***` files. Here is the list:

- `fev2flg`: embedding-tensor to flag-graph conversion - creates `.flg` and `.b.flg` files of given structure(s) out of its/their `.fev` file;
- `fev2fcs`: embedding-tensor to faces-info conversion - creates `.fcs` and `.b.fcs` files of given structure(s) out of its/their `.fev` file (as a "side effect", also creates `.flg` and `.b.flg` files);
- `flg2fev`: flag-graph to embedding-tensor conversion - creates the `.fev` file of a given structure out of its `.flg` or `.b.flg` file;
- `flg2fcs`: flag-graph to faces-info conversion - creates `.fcs` and `.b.fcs` files of a given structure out of its `.flg` or `.b.flg` file;
- `fcs2fev`: faces-info to embedding-tensor conversion - creates the `.fev` file of a given structure out of its `.fcs` or `.b.fsc` file;
- `fcs2flg`: faces-info to flag-graph conversion - creates `.flg` and `.b.flg` files of a given structure out of its `.fcs` or `.b.fsc` file;
- `for2bin`: binary-formatted conversion - in case of existing formatted `.flg` and/or `.fcs` files, creates the corresponding unformatted `.b.flg` and `.b.fcs` binary files for a given structure, and vice-versa (creates formatted files out of unformatted data). Does nothing if no formatted nor unformatted files are found.

For the `fev2flg` (`fev2fcs`) tool, if there are two or more non-equivalent flag-graphs associated with the same tensor, it will generate `.flg` and `.b.flg` files (`.fcs` and `.b.fcs` files) for all of them. In that case, the files for the first graph (structure) will be named as `systemlabel.flg` and `systemlabel.b.flg` (`systemlabel.fcs` and `systemlabel.b.fcs`), while the files for the second one will be named as `systemlabel_2.flg` and `systemlabel_2.b.flg` (`systemlabel_2.fcs` and `systemlabel_2.b.fcs`), and so on.

For `flg2fev`, there is an important note. More than being used for `.fev` to `.fev` conversion, it can be used as a tool to build the `.fev` file for a structure that you draw on a sheet of paper, for example. In your drawing, enumerate all the elements and create a "fake" `.flg` with the list of flags. For the `nface`, `neigh_flag`, and `flag_color` entries, write anything, they do not matter to build the embedding tensor. However, after using your "fake" `.flg` to generate the `.fev` file, you have to run `fev2flg` to create the corret `.flg` file with the true neighbor and orientation information.

3.3 Creating an illustrative figure of the tensor

Use the `fev2pov` tool to read a system's `systemlabel.fev` file and create a `systemlabel.pov` file for an illustration of the tensor in a perspective view. The 0/1 entries represented by red/blue cubes labelled with their values in a cartesian 3-axes system. It further illustrates the faces/edges/vertices axes in green, in a black background. The specific `systemlabel.fev` file has its `systemlabel` read from the `inp` file (containing only the `systemlabel` string in the first line).

The `.fev` format is the primary one to generate the tensor illustration. In case you do not have the `.fev` file, `fev2pov` will look for a `.b.flg` or a `.flg` file, read it, internally convert it to the tensor format and create the tensor illustration. If you do not have the `.fev` file, nor the `.b.flg` and `.flg` files, `fev2pov` will look for a `.b.fcs` or a `.fcs` file, read it, internally convert it to the tensor format and create the tensor illustration.

3.4 Computing sum rules for an embedding tensor

Use the `fev2sum` executable to compute the sum rules for an embedding tensor from a `systemlabel.fev` file and print the results in a `systemlabel.sum` file. In addition to the `systemlabel.fev` file, you have to have the `inp` file (containing only the `systemlabel` string in the first line) in the same folder.

The `.fev` format is the primary one to compute the sum rules. In case you do not have the `.fev` file, `fev2sum` will look for a `.b.flg` or a `.flg` file, read it, internally convert it to the tensor format and compute the sum rules. If you do not have the `.fev` file, nor the `.b.flg` and `.flg` files, `fev2sum` will look for a `.b.fcs` or a `.fcs` file, read it, internally convert it to the tensor format and compute the sum rules.

3.5 Identifying the symmetry group

Use the `flg2sym` executable to identify the symmetry group (printed on a `systemlabel.sym` file). You have to have the `systemlabel.flg` of `systemlabel.b.flg` file, and the `inp` file (containing only the `systemlabel` string in the first line) in the same folder. The `flg2sym` tool also produces:

- A `.map` file with the flag-symmetry maps;
- A `.rot` file with structural elements over which we have a symmetry axis, and its order;
- A `.mir` file with the structural elements unchanged under mirror reflexions, and their corresponding maps (a single element can be part of more than one mirror map).

The `.map` format contains:

- First line: the number of maps;

- Second line: the maximal number of fixed elements in the first map;
- Third line: a `map(:)` vector with size `nflags`, with `map(i)`indicating to which flag the `i`-th flag is mapped;
- Then two similar lines for the next map, ans so on.

The `.rot` format contains:

- First line: the number of element containing a rotation axis;
- Then one line for each element containing a rotation axis, with: 1) a letter (f, e, or v) indicating the type of element, 2) the index of that element, and 3) the rotation axis order.

The `.mir` format contains:

- First line: the number of element contained in a mirror;
- Then one line for each element contained in a mirror, with: 1) a letter (f, e, or v) indicating the type of element, 2) the index of that element, and 3) the index of the map it belongs.

The `.flg` format is the primary one for symmetry identification. In case you do not have the `.b.flg` or `.flg` file, `flg2sym` will look for a `.b.fcs` or a `.fcs` file, read it, internally convert it to the flag-graph format and identify the symmetry group.

3.6 Polygonal symbol generation

Use the `fcs2smb` executable to create the polygonal symbol for the studied structure. You have to have the `systemlabel.fcs` (or `systemlabel.b.fcs`) and `systemlabel.sym` files, and the `inp` file (containing only the `systemlabel` string in the first line) in the same folder. The `fcs2smb` tool produces:

- A `.smb` file with 3 versions of the symbol in latex commands;
- A pdf file for each of the 3 versions of the symbol.

The first symbol version is the simplest most sintetic one. The second includes subscripts indicating the neighboring faces of each face, but grouped when two faces of the same size have the same set of neighbors. The third is a fully extended one, with subscripts indicating the neighboring faces of each face (not grouping faces with same set of neighbors).

The `.fcs` format is the primary one for symbol generation. In case you do not have the `.b.fcs` or `.fcs` file, `fcs2smb` will look for a `.b.flg` or a `.flg` file, read it, internally convert it to the faces-info format and identify the symmetry group. Preferable, you have to run `flg2sym` first, otherwise the symbol will be generated with an x instead of the correct lattice type (t, h, c, r, or o).

3.7 Reduction to primitive cell

Suppose you have a `.flg` file that does not represent a primitive cell for a given structure. You can construct the `.fev`, `.b.flg`, `.flg`, `.b.fcs`, and `.fcs` files for the corresponding primitive cell with `flg2pri`. You have to have the `systemlabel.flg` file of your structure to make the reduction. In addition to the `systemlabel.flg` file, you have to have the `inp` file (containing only the `systemlabel` string in the first line) in the same folder. The resulting files will be named as `systemlabelprimitive.xxx` or `systemlabelprimitive.b.xxx`.

The `.flg` format is the primary one for primitive cell reduction. In case you do not have the `.b.flg` or `.flg` file, `flg2pri` will look for a `.b.fcs` or a `.fcs` file, read it, internally convert it to the flag-graph format and proceed with the primitive cell reduction.

In addition to generating the primitive cell files, `flg2pri` prints on the screen a message tellig if the input file corresponds to a primitive cell or the $N \times M$ cell size in the case of a conventional cell.

3.8 Isomorphism checking

Use the `flg2iso` tool to verify if there is an isomorphism relation between two structures. You will need the `.flg` or`.b.flg` files for the two systems and the `inp` file having the `systemlabel` strings for both systems (one per line). Upon execution, the program will print a message saying

```
Isomorphic structures.
```

or

```
Non-isomorphic structures.
```

on the screen.

The `.flg` format is the primary one for isomorphism check. In case you do not have the `.b.flg` or `.flg` file, `flg2pri` will look for `.b.fcs` or `.fcs` files, read them, internally convert them to the flag-graph format and proceed with the isomorphism check.

3.9 Structure generation

The `fcs2gen` tool creates a generation of structures with $N + 1$ faces from a parent generation of structures with N faces by means of the add dimer algorithm. In order to run it, you have to have

- A `gensizeN` file (substitute `N` in the filename by the number of faces) with the number of structures in the database of structures with N faces;

- A `gensystemsN` file (substitute `N` in the filename by the number of faces) with the systemlabel strings (one per line) for the `systemlabel.fcs` files from the database of structures of structures with N faces. In each line, after the system label, we also have three integers, indicating 1) the number of its ancestor in the previous database, 2) the face index in which the dimer was added to generate the present structure, and 3) the number of translation maps (should be 0 for primitive cell files). Note: add 1 1 0 to the graphene system in the `N=1` generation file;
- An `igen` file containing the N value of the parent generation;
- The `.fcs` files for all the structures from the database.

Once you have the correct files and the fev2kid executable, you just run it with

```
./fcs2gen
```

to generate the child generation of structures. It will generate the `.b.flg` and `.b.fcs` files for the new generation.

If you input 0 in `igen`, it will automatically generate the graphene files (labelled as `6`), together with the `gensize1` and `gensystems1` files.

3.10 Coordinates generation

During structure generation with the `fcs2gen` tool, a `.anc` file is also writen for each generated system. It contains 1) the system label of the ancestor system, 2) ancestor's face number in which the dimer is added, and 3) the indexes for the two edges within the face in which we add the dimer atoms.

Suppose we want to generate a rough initial set of coordinates for a given target structure. You can do it with the `fcs2xyz` tool. In order to use it, you will need the following files:

- An `inp` file with the system label for the target system;
- The `.b.fcs` and `.anc` files for the target system;
- The `.b.fcs`, `.xyz`, and `.nxy` files for the target's ancestor.

Then you just need to execute

```
./fcs2xyz
```

to do it.

The generated `.xyz` file (and the used ancestor's one) contains x and y coordinates between 0 and 1 (fractional coordinates), while $z = 0$ for all the atoms. It implicitly considers a pair of lattice vectors

$$\mathbf{a}_1 = (1, 0) \quad \text{and} \quad \mathbf{a}_2 = (0, 1). \tag{3.1}$$

The `.nxy` contains blocks of 3 lines, one for each face of the structure. Each block contains:

1. The number of edges in the face (`i` labels the face);

2. The `x_in_f(i,j)` values, with `j` ranging from 1 to the number of edges in the face;

3. The `y_in_f(i,j)` values, with `j` ranging from 1 to the number of edges in the face.

To understand the meaning of `x_in_f(i,j)` and `y_in_f(i,j)`, suppose the `j`-th vertex of face `i` (`v_in_f(i,j)`) is located by the $\mathbf{r}_a$ vector, while the following vertex within the same face (according to the `v_in_f` data) is located by the $\mathbf{r}_b$ vector (both vectors with coordinates within 0 to 1). However, if we eventually cross the $\mathbf{a}_1/\mathbf{a}_2$ cell when moving from the `v_in_f(i,j)` vertex to the next one within the same (`i`) face, then we move from $\mathbf{r}_a$ to $\mathbf{r}_b + x\mathbf{a}_1 + y\mathbf{a}_2$, with x being given by `x_in_f(i,j)`, and y given by `y_in_f(i,j)`.

The `fcs2xyz` tool takes the ancestor's coordinates, adds an atom in the center of each ot the edges defining the added dimer, and performs a very simple steepest descent geometry optimization along a set of steps. From the beginning, we know what are the neighbors of each atom and this information does not change (non-reactive optimization). For each geometry step, we calculate the "force" F over each atom, and promote a small displacement given by the force multiplied by a "timestep" dt. The displacement is limited by a tenth of the average distance d_0 between neighbors in the system (updated in each geometry step). Such an optimization considers three kinds of forces.

1 - *Elastic bond stretching forces:* a Hooke's law like force given by

$$\mathbf{F}_i = -k\frac{(d-d_0)}{d_0}\mathbf{u}_{ij}, \tag{3.2}$$

where

$$\mathbf{u}_{ij} = \frac{\mathbf{r}_i - \mathbf{r}_j}{d} \tag{3.3}$$

is the unit vector pointing from $\mathbf{r}_j$ to $\mathbf{r}_i$, $\mathbf{r}_i$ is the position of the atom feeling the force, $\mathbf{r}_j$ is its neighbor position, $d = |\mathbf{r}_i - \mathbf{r}_j|$ is the distance between them, d_0 is the average neighbor distance in the system, and k is the force constant.

2 - *Elastic bond angle forces:* a Hooke's law like force for bond angle variation. Consider the atom i located by $\mathbf{r}_i$ and its neighbors j and l, located by $\mathbf{r}_j$ and $\mathbf{r}_l$, respectively. Suppose $\mathbf{v}_j$ and $\mathbf{v}_l$ are the unit vectors orthogonal to $\mathbf{r}_j - \mathbf{r}_i$ and $\mathbf{r}_l - \mathbf{r}_i$, respectively, both pointing outside the $\widehat{jil} = \theta$ angle. These vectors are represented in Fig. 3.2.

The force over atom j will be given by given by

$$\mathbf{F}_j = -k_a\frac{(\theta-\theta_0)}{\theta_0}\mathbf{v}_j, \tag{3.4}$$

where $\theta_0 = 120°$ is the reference sp^2 bond angle for graphene and k_a the force constant. The force over atom l will be given by

$$\mathbf{F}_l = -k_a\frac{(\theta-\theta_0)}{\theta_0}\mathbf{v}_l, \tag{3.5}$$

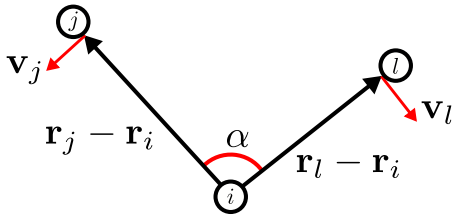

Figure 3.2: Illustration of the position $\mathbf{r}_i$ of the atom i the positions $\mathbf{r}_j$ and $\mathbf{r}_l$ of its neighbors j and l, as well as the $\widehat{jil} = \theta$ angle and the unit vectors $\mathbf{v}_j$ and $\mathbf{v}_l$ (orthogonal to $\mathbf{r}_j - \mathbf{r}_i$ and $\mathbf{r}_l - \mathbf{r}_i$, respectively) pointing outside the $\widehat{jil} = \theta$ angle.

and the force over atom i will be given by

$$\mathbf{F}_i = -\mathbf{F}_j - \mathbf{F}_l. \tag{3.6}$$

3 - *Coulomb repulsive forces:* a Coulomb like force given by

$$\mathbf{F}_i = \frac{q}{d/d_0}\mathbf{u}_{ij}, \tag{3.7}$$

where q is the "charge" force constant, and $\mathbf{F}_i$ the force over atom i. The atom j will feel the force $\mathbf{F}_j$ given by

$$\mathbf{F}_j = \frac{q}{d/d_0}\mathbf{u}_{ji} = \mathbf{F}_i, \tag{3.8}$$

as $\mathbf{u}_{ji} = -\mathbf{u}_{ij}$.

The important quantities for the structure optimization are k, k_a, q, and dt. These can be determined by the user through the `relax.par` file, which contains these quantities (one per line) in this sequence. The `relax.par` file still has a fith line containing an integer that tells if the program will draw a `.svg` and a `.pdf` file with an illustration of the atomic structure (0 for no, any othere integer for yes). Below we show an example for this set of parameters in the `relax.par` format.

```
1.0D0    # k
8.0D0    # ka
1.0D0    # q
0.001    # dt
10000    # nstepmax
1        # makesvgpdf
```

Depending on your system, you may have to tune these parameters to find reasonable results.

If you want to generate the coordinates for a given system, you have to have its `.anc` file and the `.xyz`, `.nxy`, and `.anc` files for its ancestor. If you do not have the coordinates for the ancestor, the program looks for the ancestor's ancestor, and so on, down to graphene (which is the only system with two atoms). This works if you have

run `fcs2gen` and have the files for the previous generations of structures in the working directory. Then, when it finds an ancestor of the tree with already generated `.xyz` and `.nxy` files, it generates the coordinates through the following generations until the target system. Even if you do not have the `.xyz` and `.nxy` files for graphene, the program will create them in the last scenarion. Even though creating these files for graphene is not a hard task with a little thinking and maybe a drawing on a piece of paper. Below we show a possible form for graphene's `.xyz` and `.nxy` files.
`graphene.xyz`:

```
2

C  0.37  0.37  0.00
C  0.67  0.67  0.00
```

`graphene.nxy`:

```
6
0  0 -1  0  0  1
0  1  0  0 -1  0
```

It it worth to remember that the `.fev` and `.nxy` have very simple forms, as represented below.
`graphene.fev`:

```
1    3    2
1
1
1
1
1
1
```

`graphene.fcs`:

```
1    3    2    6
6  T
1      1      1      1      1      1
1      2      3      1      2      3
1      2      1      2      1      2
T      T      T      T      T      T
T      F      F      F      F      F
```

If the target system listed on the `inp` file is labelled as `graphene` or simply `6`, the `fcs2xyz` tool will produce sample `.fev`, `.flg`, `.fcs`, `.b.fcs`, `.nxy`, and `.xyz` files for the graphene structure, as well as a `.svg` and a `.pdf` with the corresponding structure drawing.

Below we reproduce the pdf images generater for structures with up to 3 faces.

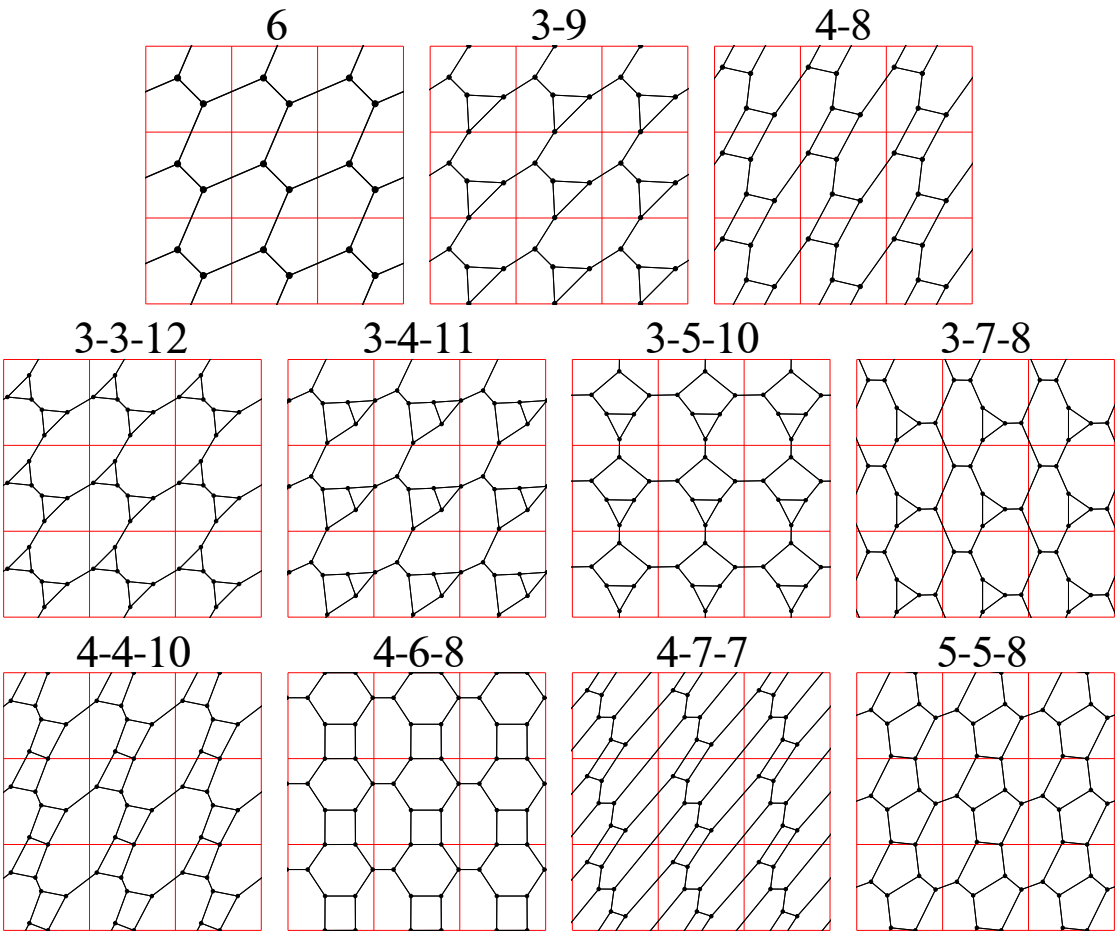

Figure 3.3: Output pdf images for the coordinates generated for the systems with up to 3 faces.

3.11 Files generation from coordinates

Suppose you have the `.xyz` file with the coordinates of a system and a `.lat` file with the two lattice vectors, one per line. You can use `flg4xyz` to generate the corresponding `.fev`, `.b.flg`, `.flg`, `.b.fcs`, `.fcs` files for that system. This is an alternative way to generate files for a particular system by using `\flg2fev`.